Plano de Estudos SRE: Trilhas de Aprendizado por Nível

Por Tharlesson

Este guia apresenta trilhas de aprendizado separadas para **SRE Júnior**, **Pleno**, **Sênior** e **Especialista**, focando em conteúdos **gratuitos, online e em português**. Cada trilha inclui habilidades esperadas no nível e uma lista de conteúdos recomendados (cursos, vídeos, guias, artigos, labs) organizados por tema (Fundamentos de SRE, Tecnologias, Observabilidade, Pipelines). Aproveite os materiais sugeridos para desenvolver competências em **SLI/SLO/Error Budgets, automação, confiabilidade, resposta a incidentes**, além de ferramentas como **Linux, Docker, Kubernetes, AWS, Terraform, Ansible, Prometheus, OpenTelemetry, New Relic e Jenkins**.
Mas nada te impede de sugerir e utilizar outras ferramentas como Karpenter, ArgoCD, Github Actions, ... , etc.

Como usar este Plano: Cada profissional pode seguir a trilha do seu nível, mas é útil olhar também o nível seguinte para planejar seu desenvolvimento. As seções de tecnologias e observabilidade são cumulativas – um SRE Sênior deve dominar o conteúdo dos níveis anteriores e assim por diante. Os links fornecidos apontam para materiais gratuitos de qualidade (cursos, artigos, documentações) em português, para facilitar o aprendizado. Bons estudos e *Site Reliability Engineering* feliz!

Referências Utilizadas: Plano de Estudos SRE: Trilhas de Aprendizado por Nível | SRE Júnior Fundamentos e Primeiros Passos

- Stone Co. *Trilha de Carreira SRE* – descrevendo responsabilidades e desafios por nível (SRE I a Principal) , o que guiou as habilidades esperadas em cada nível.
- Artigos técnicos (Medium, Atlassian, New Relic) em português sobre SLI/SLO/SLA e OpenTelemetry .
- Conteúdos educacionais gratuitos citados: Curso em Vídeo Linux , Livro “Descomplicando Docker” (aberto via GitHub) , Treinamento “Descomplicando Kubernetes” (LINUXtips) , curso AWS Cloud Practitioner (Grupo Boticário/AWS) , guia Terraform em PT , curso Jenkins Udemy

grátis , curso Prometheus/Grafana , entre outros. Todas as fontes são de acesso livre conforme indicado nos respectivos links.

SRE Júnior

Habilidades e Conhecimentos Esperados

- **Visão Geral**: No nível Júnior, espera-se atuação em tarefas de escopo bem definido, com orientação de colegas mais experientes, focando principalmente em **aprender os fundamentos** da operação de sistemas e práticas de SRE . O SRE Júnior contribui trabalhando dentro de sua equipe, seguindo processos estabelecidos, enquanto desenvolve conhecimentos básicos de infraestrutura, automação e depuração de problemas simples.
- **Fundamentos de SRE**: Deve compreender os conceitos básicos de **confiabilidade, disponibilidade e monitoramento**, e familiarizar-se com termos como **SLA, SLI, SLO e orçamento de erros** (error budget). Por exemplo, saber que *SLI* é um indicador de nível de serviço (ex.: taxa de erro, latência) e *SLO* é a meta de desempenho acordada para esses indicadores . Também é importante entender a cultura SRE de balancear risco com entregas frequentes e automatizar tarefas operacionais repetitivas.
- **Autonomia**: Espera-se que o júnior realize tarefas seguindo roteiros e scripts já definidos, pedindo ajuda quando necessário. O principal impacto de seu trabalho é **aprender e aprimorar suas habilidades técnicas** para evoluir na carreira .

Fundamentos de SRE (Teoria e Prática)

- **Curso Introdutório de SRE – Google Cloud (Webinar)** – *Introdução ao Site Reliability Engineering (SRE)* em português (Google Cloud). Webinar gratuito que apresenta os princípios básicos de SRE e a cultura do Google nessa área (conceitos de SLO, automação, etc.). *(Vídeo no YouTube, Google Cloud Community)*.
- **Leitura: Livro Site Reliability Engineering (Google)** – Tradução oficial para português do famoso livro de SRE do Google, disponível online gratuitamente . Este livro cobre desde os princípios (embrace risk, eliminando trabalho manual) até práticas de implementação de

SLOs, gerenciamento de incidentes e cultura postmortem. Excelente referência para construir fundamentos sólidos em SRE. *Disponível via ElvenWorks, capítulos em português* .

- **Artigo: SLI, SLO, SLA e Error Budget** – Explicação em português desses conceitos de confiabilidade, com exemplos práticos . Ajuda o iniciante a entender como medir a saúde de serviços e estabelecer metas de nível de serviço.
- **Guia Prático de SRE (ElvenWorks)** – Curso gratuito de formação básica em SRE/DevOps pela Elven Works . Cobre fundamentos de Cloud e confiabilidade, ideal para quem está iniciando (inscrição na plataforma pode ser necessária).
- **Curso: Developing a Google SRE Culture** – curso online (Google/Coursera) em português. Apresenta as principais práticas de SRE do Google e fundamentos técnicos/culturais para reduzir silos e melhorar confiabilidade . *Gratuito para assistir como ouvinte na Coursera.*

Tecnologias Essenciais (Linux, Docker, Kubernetes, AWS, Terraform, Ansible)

- **Linux (Sistemas Operacionais)**: Iniciar pelos fundamentos do Linux, já que SREs trabalham majoritariamente em ambientes Linux. Recomenda-se o *Curso "Linux – Primeiros Passos" (Curso em Vídeo)* – uma série gratuita e em português, com 14 aulas práticas cobrindo desde instalação do Linux até comandos básicos de terminal . Este curso oferece base sólida em Linux para quem vem de Windows e pretende administrar servidores Linux.

  - *Material*: Playlist **"Curso de Linux – Primeiros Passos"** (Canal Curso em Vídeo/Gustavo Guanabara). Inclui histórico do Linux, comandos de shell, manipulação de arquivos e permissões, montagem de dispositivos, etc. .

- **Docker (Contêineres)**: Aprender contêineres é essencial. Uma ótima referência é o livro online **"Descomplicando o Docker"** (Jeferson Fernando, LINUXtips), disponibilizado gratuitamente . Esse material abrange Docker do nível iniciante ao avançado, ensinando a criar containers, imagens, usar Dockerfile, Docker Compose e Swarm.

  - Para algo mais rápido, há o **"Intensivão Docker em 2h"** (vídeo YouTube) – curso gratuito condensado em português explicando o básico de Docker (conceitos de imagem, container, comandos fundamentais).
  - *Outra opção*: **Curso Docker Fundamentals (DIO)** – curso prático gratuito na plataforma DIO cobrindo criação e gerenciamento de containers Docker em atividades reais .

- **Kubernetes (Orquestração de Contêineres)**: Após Docker, estudar Kubernetes. Recomenda-se o conteúdo aberto **"Descomplicando o Kubernetes"** (LINUXtips) – treinamento completo em português, originalmente dado ao vivo e agora disponível como livro online gratuito . Cobre do básico (arquitetura do k8s, instalar **kubectl** e criar clusters locais com Minikube/Kind) até conceitos avançados (Deployments, Services, ConfigMaps, etc.), em uma trilha de aprendizado dia-a-dia.

  - *Dica*: A estrutura "Descomplicando Kubernetes" é dividida em 16 dias de conteúdo, permitindo ao aprendiz evoluir passo a passo . Cada dia inclui exercícios práticos, o que é ótimo para fixar conhecimentos de **Pods, ReplicaSets, Deployments, networking no Kubernetes** e gestão de clusters.
  - Além do livro, a LINUXtips liberou vídeos correspondentes no YouTube (procure por *"Descomplicando o Kubernetes"* no canal LINUXtips).

- **AWS (Serviços de Nuvem)**: Comece pelos fundamentos de computação em nuvem na AWS. Uma recomendação é o treinamento **AWS Cloud Practitioner Essentials** em português, disponível gratuitamente no site da AWS Training . Esse curso aborda os principais serviços (EC2, S3, RDS, etc.), modelo de responsabilidade compartilhada, noções de arquitetura e precificação – conhecimentos importantes para um SRE júnior entender infraestrutura em nuvem.

  - **AWS Educate** e **AWS Skill Builder**: plataformas gratuitas da AWS (em português) com trilhas de aprendizado iniciais em Cloud. Incluem laboratórios práticos sem custo . Inscreva-se para acessar módulos em português sobre conceitos como escalabilidade, alta disponibilidade e uso básico do console AWS.
  - *Conteúdo complementar*: Webinar *"Introdução à AWS"* (AWS Brasil Oficial) – vídeo em português que demonstra na prática como criar servidores EC2, configurar auto scaling, etc., dando uma noção prática dos serviços cloud.

- **Terraform (Infraestrutura como Código)**: Para gestão de infraestrutura automatizada, aprenda Terraform. Recursos sugeridos:

- **Guia Rápido Terraform em Português** – tutorial de sobrevivência escrito por Edson Célio . Explica o que é Terraform e como começar, incluindo definição do HCL (Hashicorp Configuration Language).
- **Cursos gratuitos no YouTube**: há playlists recomendadas, como *"Terraform 1.0 – do básico a módulos"* por Caio Delgado, e *"Terraform 101 – Instalação e deploy na AWS"* . Esses vídeos em português(BR) conduzem o iniciante na instalação do Terraform, criação de recursos AWS simples e introdução a módulos.
- **Repositório "Terraform para Iniciantes"** – curso gratuito compartilhado na comunidade (por exemplo, via Reddit) com exercícios práticos passo-a-passo (procurar no GitHub ou posts do Medium).
- Após o básico, consultar **boas práticas**: O guia *"Terraform Best Practices"* (há versão PT-BR disponível) compila recomendações de organização de código Terraform, padrões de módulos e workflows de equipe – útil quando o SRE júnior for evoluindo.

- **Ansible (Automação e Configuração)**: Ansible é uma ferramenta de automação de configuração muito usada por SREs para gerenciar servidores. Conteúdos gratuitos:

  - **Curso "Descomplicando o Ansible"** – treinamento completo da LINUXtips que foi disponibilizado gratuitamente (vídeos ~12h no YouTube). Ensina do zero: instalação, escrever *playbooks*, usar módulos para configurar máquinas e até provisionar clusters. *(Ver no YouTube: "Descomplicando o Ansible – Completo" – conteúdo em português)* .
  - **Tutorial Ansible para Iniciantes (Red Hat)** – a Red Hat oferece um tutorial interativo gratuito do Ansible Automation Platform , cobrindo fundamentos de playbooks, inventário, etc., em português. Além disso, laboratórios online da Red Hat estão disponíveis sem custo para prática.
  - **Curso gratuito de Fundamentos do Ansible** – curso em formato de posts curtos (disponível via Reddit ou blogs) criado por membros da comunidade, ótimo para quem prefere aprender lendo em português .

Observabilidade e Monitoramento (Prometheus, OpenTelemetry, New Relic)

- **Prometheus (Métricas e Monitoramento)**: Ferramenta open-source essencial para SREs monitorarem sistemas.

- Recomenda-se o **Curso de Monitoramento com Prometheus e Grafana** (pt-BR) por Ciro Lini, disponível no GitHub . Este curso gratuito ensina desde a instalação do Prometheus (local ou em Kubernetes) até consulta de métricas (PromQL), setup do Alertmanager para alertas e criação de dashboards no Grafana . O sumário do curso cobre tópicos práticos: exporters (como node_exporter, cadvisor), alertas preditivos (usando funções como predict_linear), federar múltiplos servidores Prometheus e integração com Kubernetes . É um material bem completo para adquirir experiência em observabilidade de infraestrutura.
- **Video Aula: Introdução ao Prometheus e Grafana** – disponível no YouTube (em português), mostrando a configuração básica do Prometheus coletando métricas de um app e visualização no Grafana.
- **Documentação Prometheus** – Embora esteja em inglês, vale consultar a seção "Getting started" no site oficial e a documentação do **PromQL** (consultas) quando for necessário aprofundar.

- **OpenTelemetry (Tracing e Telemetria unificada)**: Padrão aberto para instrumentação de aplicações (tracing distribuído, métricas e logs). Para introdução:

  - Leia a seção **"Introdução à Observabilidade"** do site OpenTelemetry em português . Ele explica o que é observabilidade (capacidade de entender o sistema pelas métricas, logs e rastros emitidos) e como o OpenTelemetry é utilizado para instrumentar código e coletar esses sinais. Por exemplo, a doc destaca que uma aplicação instrumentada expõe *traces*, *métricas* e *logs* sem precisar alteração de código a cada novo problema .
  - **Artigo: "O que é OpenTelemetry? (New Relic)"** – publicação em português detalhando o projeto OpenTelemetry e seus benefícios . Explica de forma didática por que o OpenTelemetry está se tornando o padrão de instrumentação (APIs unificadas para várias linguagens, não atrelado a fornecedor, suporte a *distributed tracing*, etc.) . É útil para compreender conceitos de tracing distribuído e contexto de trace em sistemas complexos.
  - **Vídeo: Implementando Observabilidade com OpenTelemetry** – palestra em português (YouTube) mostrando na prática como instrumentar um serviço com OpenTelemetry (capturando traces de requisições e exportando para uma ferramenta).

- **New Relic (APM e Observabilidade de aplicações)**: Plataforma popular de monitoramento de desempenho de aplicações. Para SREs juniores, é interessante conhecer ferramentas APM:

- **Documentação New Relic (Português)** – A New Relic oferece parte de sua documentação e blog em português. Por exemplo, o artigo *"O que é OpenTelemetry? O futuro da instrumentação"* no blog da New Relic (já citado acima) também introduz conceitos gerais de observabilidade unificada que a plataforma suporta .
- **New Relic University** – disponibiliza treinamentos gratuitos (em inglês, alguns legendados) sobre como usar a plataforma. Você pode criar uma conta gratuita ("free tier") na New Relic para praticar monitoramento de uma aplicação demo – é uma forma de aprender a definir alertas de APM, gráficos de métricas customizadas e análise de traces.
- **Guia: "Observabilidade para desenvolvedores"** (New Relic blog pt-BR) – artigo que discute boas práticas de instrumentação de código com telemetria (como inserir logging estruturado, métricas de negócio e spans de tracing). Ajuda o iniciante a entender **o que instrumentar** em sua aplicação para torná-la observável.

Pipelines e CI/CD (Jenkins)

- **Jenkins (Integração Contínua/Entrega Contínua)**: Aprender a criar pipelines automatizados de build, test e deploy:

  - **Curso "Jenkins Beginner Tutorial – Step by Step"** – curso gratuito de Jenkins para iniciantes, oferecido na Udemy (instrutor *Raghav Pal*). Ensina do zero: instalação do Jenkins, configuração inicial, integração com GitHub e criação de pipelines básicas. O curso está **disponível gratuitamente** em português para quem se inscrever . Ao final, o aluno será capaz de usar o Jenkins em projetos pessoais ou profissionais, implementando integração contínua e entrega contínua.

    - *Dica*: O Grupo de Profissionais Oracle (GPO) divulgou esse curso – basta ter uma conta Udemy e **clicar no link de acesso gratuito** fornecido no artigo .

  - **Playlist "Curso de Jenkins" (YouTube)** – série de vídeos curtos em português cobrindo tópicos específicos: por exemplo, *Instalação do Jenkins* (no Windows ou Linux), *Criando primeiro job*, *Integração do Jenkins com Git*, *Configuração de Pipeline as Code*

(Jenkinsfile), *Automatizando testes* etc. Esses vídeos permitem aprendizado incremental e prático.

- **Documentação do Jenkins em Português** – O site oficial do Jenkins possui uma parte traduzida para PT-BR com tutoriais básicos. Consulte o *Tutorial de Pipeline (Declarative Pipeline)* para entender como escrever pipelines Jenkinsfile declarativos, e a seção de *Blue Ocean* (UI visual de pipelines) para facilitar a visualização.
- **Labs Práticos**: Utilize cenários gratuitos como *Katacoda* ou *Play with Docker* que têm módulos de Jenkins – por exemplo, há um lab de CI/CD com Jenkins e Docker disponível, guiando pela criação de um pipeline que faz build de container e implantação.

SRE Pleno

Habilidades e Conhecimentos Esperados

- **Visão Geral**: O SRE Pleno já possui experiência e **maior autonomia**. É capaz de entregar soluções de valor com escopo bem definido e parcialmente refinadas, precisando de menos orientação em tarefas cotidianas . O foco neste nível é **consolidar habilidades técnicas**, ganhar confiança e começar a ter visão de design de sistemas. O pleno contribui implementando melhorias consistentes nos sistemas existentes para aumentar confiabilidade e eficiência da plataforma .
- **Fundamentos de SRE**: Espera-se domínio prático dos conceitos SRE. O pleno consegue **definir e medir SLOs**dos serviços sob sua responsabilidade e entender implicações de error budgets na priorização de tarefas. Também participa ativamente de **resposta a incidentes**, podendo liderar postmortems de menor complexidade. Deve saber identificar *trade-offs* entre velocidade de lançamento e estabilidade, ajudando a manter o equilíbrio correto.
- **Escopo e Impacto**: Atua principalmente dentro de sua equipe, mas já colabora com outras equipes em projetos interdependentes. O SRE pleno começa a contribuir em **desenho de componentes de sistemas** e traduz requisitos de confiabilidade em projetos simples de engenharia . Por exemplo, pode projetar scripts de automatização mais complexos, pipelines de implantação ou melhorias na infraestrutura que reduzem toil. Também pode **mentorar juniores** em tarefas básicas e boas práticas .
- **Conhecimentos Técnicos**: Além de aprofundar-se nas tecnologias listadas para juniores, o pleno deve conhecer **arquiteturas tolerantes a falhas** em alto nível (conceitos de redundância, replicação, failover) e começar a se envolver na implementação dessas melhorias. Entende e depura problemas de performance, identificando gargalos em sistemas

distribuídos . Também deve estar confortável com programação básica ou scripting para criar ferramentas internas e automatizar processos end-to-end.

Fundamentos de SRE (Teoria e Prática)

- **Livro Site Reliability Workbook (Google)** – Continuação prática do livro SRE, também traduzido PT-BR gratuitamente . Este workbook traz estudos de caso e orientações de implementação (por ex: como iniciar um projeto de SLO, criar playbooks de incidente, etc.). SREs Pleno devem estudar capítulos relevantes como **Definição de SLOs** na prática, **Gestão de Incidentes** e **Eliminação de Toil**. *Disponível via ElvenWorks (tradução livre online)*.
- **Error Budgets e Equilíbrio Dev x Ops** – Aprofundar no conceito de *embracing risk*. Recomenda-se a leitura do capítulo *"Objetivos de Nível de Serviço (SLO) e Orçamento de Erros"* do livro SRE (Capítulo 4) . Este capítulo explica como estabelecer SLOs e derivar o error budget, além de estratégias de negociação com times de desenvolvimento quando o error budget é consumido.
- **Curso SRE Intermediate (ElvenWorks)** – Verifique se a ElvenWorks ou outras plataformas oferecem um módulo intermediário (algumas empresas oferecem formações SRE avançadas gratuitas). Por exemplo, um *Programa de Formação SRE Avançado* cobrindo *redução de toil, SRE tools*, etc. Caso disponível, aproveite.
- **Google Cloud Skills Boost – SRE**: Laboratórios práticos gratuitos no Qwiklabs/Cloud Skills Boost focados em SRE. Ex.: *"Quantificando riscos para SLOs"* (lab em português) , onde se pratica o cálculo de error budgets e monitoração de SLO no Google Cloud. Esses labs ajudam o pleno a aplicar teoria de SLO em cenários reais.
- **Artigo: Boas Práticas de SRE** – Atlassian Pt-BR *"SLA vs SLO vs SLI: qual a diferença?"* . Como pleno, é importante garantir que esses conceitos estejam claros para orientar times. O artigo resume como SLAs são promessas ao cliente, SLOs metas internas, e SLIs medições rastreáveis – e discute *práticas recomendadas* de não complicar com métricas demais .

Tecnologias Essenciais (Linux, Docker, Kubernetes, AWS, Terraform, Ansible)

- **Linux (Avançado)**: Aprofundar conhecimentos de administração e tunning. Conteúdos:
  - **Curso "Linux do Zero ao Engenheiro DevOps" (Cisco/NDG)** – disponível gratuitamente via Cisco Networking Academy (Linux Unhatched/Essentials em português). Cobre tópicos como gerenciamento de usuários, processos, rede, e fornece labs interativos.

- **Livro/Guia Foca Linux** – material clássico em português (PDF gratuito) abordando comandos avançados, shell scripting e conceitos de kernel e serviços de sistema. Útil para referência quando precisar otimizar um sistema ou escrever scripts complexos.
- **Performance e Debug**: Artigo "Resolvendo problemas de performance no Linux" (em algum blog PT) ou vídeos do canal Bóson Treinamentos sobre *Monitoramento de recursos (htop, iostat, etc.)*. SREs plenos devem conseguir investigar alta carga de CPU, consumo de memória, problemas de IO e aplicar soluções (ajustes de config, sysctl tuning).

- **Docker/Containers (Avançado)**: Além do uso básico, aprender:

  - **Docker Internals** – entender conceitos de cgroups e namespaces do Linux por trás do Docker. Por exemplo, há palestras ("Como o Docker funciona por baixo dos panos") em português que explicam essas bases.
  - **Imagens Otimizadas** – conteúdos sobre criação de imagens Docker eficientes (multi-stage builds, reduzir tamanho de imagem) – blog posts em português da comunidade (ex: *"Boas práticas com Dockerfile"* no Medium BR).
  - **Security**: Artigo em português sobre segurança em containers (uso de *Docker Bench for Security*, limitar capabilities, etc.), pois SRE pleno já deve se preocupar com hardening do ambiente containerizado.

- **Kubernetes (Avançado)**: Focar em operação de clusters:

  - **Livro Kubernetes – Descomplicando (Portugues)** – complementar ao curso, se disponível. A LINUXtips liberou materiais até nível *Senior Staff SRE*, então verifique se há capítulos avançados no *Descomplicando Kubernetes* sobre *deploy canário, operadores, custom resource definitions (CRDs)* etc.
  - **Labs Kubernetes**: Utilizar playgrounds (Katacoda scenario *"Kubernetes Hardway (PT)"* se existir, ou labs da CNCF) para praticar configurações avançadas: deployment rollout strategies, liveness/readiness probes customizadas, utilização do **Prometheus Operator** no cluster, etc.
  - **Certificação Kubernetes (CKA/CKAD)**: Mesmo que não faça a prova, estudar os tópicos das certificações ajuda. Há apostilas em português criadas por profissionais certificados, muitas vezes gratuitas, que cobrem gerenciamento de nós, backup/restore *etcd*, network

policies, volumes, etc. SRE Pleno deve ser capaz de **gerenciar e solucionar problemas** em clusters Kubernetes de produção.

- **AWS (Intermediário/Arquitetura)**: Expandir conhecimento para desenhar soluções confiáveis:

  - **Curso AWS Solutions Architect Associate (em português)** – diversos canais oferecem cursos preparatórios grátis. Por exemplo, o canal *"AWS Brasil (Comunidade)"* no YouTube tem uma série de lives cobrindo os domínios da certificação SAA-C03 em português. Ao acompanhar, o SRE pleno aprende a **arquitetar sistemas escaláveis e tolerantes a falhas na AWS** (usando Auto Scaling, Multi-AZ RDS, S3 Durabilidade, etc.).
  - **Workshops AWS Well-Architected (pt-BR)** – A AWS disponibiliza workshops práticos com cenários de arquitetura. Procure a versão traduzida ou legendada do *Well-Architected Labs*. Isso ajuda a treinar a mentalidade de arquitetura (pilares de *Reliability* e *Performance Efficiency*).
  - **Serviços específicos**: SRE pleno deve conhecer além do básico: VPC e redes (peering, VPN), serviços de mensageria (SQS, SNS), observabilidade na AWS (CloudWatch, X-Ray). A AWS possui **webinars gratuitos em português** sobre muitos desses assuntos (ex.: *"Melhores práticas de CloudWatch"*).

- **Terraform (Intermediário/Team)**: O pleno deve ser capaz de estruturar código Terraform para projetos maiores e colaborar com equipes:

  - **Módulos e Reutilização**: Ler o capítulo de *módulos* no guia Terraform oficial. E usar repositórios open-source de módulos (Terraform Registry) em português, se disponíveis, para estudar como escrever módulos reutilizáveis e seguros. Por exemplo, o **repositório terraform-aws-modules** (embora documentado em inglês) é bom para entender padrões.
  - **Estado Remoto e Colaboração**: Aprender a configurar *remote state* (em backends como S3 + DynamoDB para lock) – há tutoriais em português no blog da Hashicorp ou comunidade explicando passo a passo. Isso é crucial para trabalho em equipe.
  - **Ferramentas complementares**: Conhecer *Terragrunt* (há conteúdo PT citando quando usar Terragrunt para organizar ambientes) e ferramentas de validação estática (como *tflint*, *tfsec*) – muitas citadas no *Breve Guia de Terraform* . SRE pleno deve começar a incorporar essas práticas de qualidade no código IaC.

- **Ansible (Intermediário)**: Com bases sólidas, o pleno pode:

  - Escrever **roles Ansible** modularizadas. Seguir o padrão *Ansible Galaxy* (há artigos pt-BR sobre como estruturar playbooks grandes em múltiplos roles).
  - Integrar Ansible em pipelines CI/CD – por exemplo, usar Ansible para deploy contínuo. Um caso prático: **GitLab CI com Ansible** – tutoriais comunitários demonstram como rodar playbooks automaticamente após testes. Tais conteúdos ajudam o SRE a orquestrar infra as code dentro do pipeline.
  - Explorar **Ansible Automation Platform**: a Red Hat tem webinars gratuitos em português mostrando features avançadas (AWX/Ansible Tower para controle centralizado, workflows de job, credenciais). Isso dá visão de como escalar automações Ansible em ambientes corporativos.
  - *Dica de conteúdo*: Série de posts *"Ansible Avançado"* no blog "4Linux" ou similar, abordando tópicos como *Ansible Vault* (segredos), *Dynamic Inventory* (integrando com AWS, etc.), e otimizações (strategies, forks, callback plugins).

Observabilidade e Monitoramento (Prometheus, OpenTelemetry, New Relic)

- **Prometheus (Escala e Alertas Avançados)**: O SRE Pleno deve ser capaz de montar uma stack de monitoramento robusta:

  - **Gráfico de Arquitetura Prometheus** – Estudar (no curso citado) a seção de *federation* e *scraping de múltiplas fontes*. Aplicar esse conhecimento em um lab: ex., configurar um Prometheus federado que reúne métricas de vários clusters, garantindo escalabilidade de monitoramento.
  - **Alerting avançado**: Aprender a definir *alertas baseados em sintoma e não em causa*. Por exemplo, alertar em erro do usuário final ou saturação percebida, em vez de cada minúcia – conforme as recomendações do Google SRE. O *capítulo 6 do livro SRE ("Monitorando Sistemas Distribuídos")* é uma boa leitura complementar . Em português, buscar materiais sobre **"Four Golden Signals"** (latência, tráfego, erros, saturação) e implementar painéis cobrindo esses sinais.

- **Ferramentas relacionadas**: Conhecer e usar **Grafana** proficientemente (criando dashboards customizados, usando alertas via Grafana Alerting). Também experimentar o **Alertmanager** integrando com canais como PagerDuty, Slack (muitos blogs PT explicam configuração do Alertmanager com exemplos práticos).
- **Exporters customizados**: SRE pleno pode escrever um **exporter simples**. O curso do Ciro Lini mostra como criar seu próprio exporter em Python/Go – é um ótimo exercício para entender profundamente métricas e instrumentação.

- **OpenTelemetry (Aprofundando e Integrando)**:

  - **Implementação prática**: Instrumentar um serviço real com OpenTelemetry. Por exemplo, pegar uma aplicação demo (como uma aplicação Node.js ou Java) e utilizar as bibliotecas OpenTelemetry PT-BR para coletar traços e métricas. A documentação em português do OpenTelemetry fornece guias para várias linguagens .
  - **Collector**: Aprender a configurar o **OpenTelemetry Collector** – componente que recebe dados e exporta para sistemas (New Relic, Prometheus, etc.). O site OpenTelemetry tem guia em PT sobre instalação do Collector e pipelines de processamento.
  - **Case de uso**: Ler um case público (em português) de adoção do OpenTelemetry. Ex: artigo *"Implementando Observabilidade com OpenTelemetry na Empresa X"* se disponível em comunidades (ou assistir palestra legendada de conferência). Isso dá insight de desafios e benefícios práticos.
  - **Ferramentas de Trace**: Como pleno, experimente diferentes backends: por exemplo, enviar dados OpenTelemetry para o **Jaeger** (ferramenta open-source de tracing) ou **Zipkin**, além do New Relic. Assim entenderá o formato padrão e como é analisado em diversas plataformas.

- **New Relic e APM (Avançado)**:

  - **APM avançado**: SRE pleno deve conseguir criar **painéis customizados (dashboards)** no New Relic com métricas de negócio e técnicos, escrever **NRQL queries** simples para obter insights não cobertos pelos gráficos padrão, e configurar **alert conditions** sofisticadas (ex.: alerta se taxa de erro sobe acima de X *e*latência média acima de Y). A New Relic University oferece vídeos (em inglês) sobre criação de alertas e dashboards – utilize legendas em português se necessário.

- **Infraestrutura e Logs**: Aprender a usar o **New Relic Infrastructure** (monitoramento de hosts, integrando com Prometheus/OpenMetrics) e **New Relic Logs**. Há tutoriais em português no blog da New Relic ou canais como Medium mostrando, por exemplo, *"Monitorando Kubernetes com New Relic"* ou *"Centralizando logs com New Relic"*.
- **Comparar ferramentas**: Para não ficar restrito a um fornecedor, é útil conhecer similares. Talvez testar o **Datadog** ou **Elastic Stack (ELK)** em versões free/open. Embora a pergunta foque New Relic, um SRE pleno ganha analisando diferentes soluções de observabilidade (muitas têm material PT-BR introdutório). Isso amplia a visão crítica sobre requisitos de monitoramento e observabilidade.

Pipelines e CI/CD (Jenkins e além)

- **Jenkins (Pipelines como Código e Escalabilidade)**: Pleno deve dominar automações CI/CD:
  - **Pipeline as Code (Jenkinsfile)**: Aprender a escrever pipelines declarativos e *scripted* no Jenkins. O *Jenkins Beginner Tutorial* já introduz, mas agora é hora de aprofundar: usar *stages paralelos*, pós-acoes (post steps) para notificação, e bibliotecas compartilhadas do Jenkins. A documentação oficial (en-US) é referência; para PT, alguns blogs explicam pipeline Jenkinsfile passo a passo.
  - **Jenkins Avançado**: Explorar plugins populares: *Blue Ocean* (visual pipeline), *Pipeline:Multibranch* (para CI multi-branch), *Credentials Binding*, *Matrix builds*, etc. Identifique necessidades e busque artigos PT ou discussões no Stack Overflow em português para dúvidas específicas.
  - **Infraestrutura do Jenkins**: Entender como escalar Jenkins com *agents (slaves)*. Há conteúdos em português sobre configurar Jenkins em modo distribuído e usar agentes em Docker Kubernetes (por ex.: *"Jenkins em Kubernetes"* webinar PT). SRE pleno deve ser capaz de manter a ferramenta CI funcionando confiável, o que inclui backup do Jenkins, monitoramento dos executores e otimização (limpeza de workspaces, etc.).
  - **Alternativas e Ferramentas CI/CD**: Embora Jenkins seja citado, um SRE pleno deve ao menos ter noção de outras ferramentas de mercado:
    - **GitLab CI**: plataforma integrada que muitos projetos usam – a documentação do GitLab possui guia em português. Entender a sintaxe .gitlab-ci.yml e compará-la com Jenkinsfile dá perspectiva.

    - **GitHub Actions**: se sua empresa/projeto usa GitHub, conhecer Actions é útil. Existe documentação oficial PT-BR e blogs como *"CI/CD com GitHub Actions – guia prático"* em português.
    - **ArgoCD/Tekton** (opcional, se houver interesse em CI/CD cloud-native): Há webinars em português sobre ArgoCD (Continuous Delivery para K8s declarativo).

  - **Melhores Práticas de Pipeline**: Ler artigos ou whitepapers (PT se possível) sobre *"CI/CD pipelines eficientes"* – cobrindo tópicos como pipeline trunk-based, estratégias de deployment (blue/green, canary) e teste contínuo. Um SRE pleno pode não implementar tudo isso sozinho, mas deve conhecer para colaborar com Devs na melhoria do processo de release.

SRE Sênior

Habilidades e Conhecimentos Esperados

- **Visão Geral**: O SRE Sênior atua como **referência técnica** no time e frequentemente além dele. É capaz de identificar soluções ótimas equilibrando prazo e qualidade para **problemas complexos e ambíguos**, geralmente sem escopo definido previamente . Ou seja, lida com desafios inéditos, define escopos e direciona a equipe na resolução. O sênior tem **pensamento estratégico**: suas decisões consideram impactos de médio a longo prazo em confiabilidade e negócios.
- **Liderança Técnica**: Espera-se que o SRE Sênior **lidere projetos de infraestrutura/fiabilidade de maior porte** de forma autônoma . Ele define roteiros técnicos, frequentemente coordenando esforços entre múltiplas equipes. Também exerce forte papel de **mentoria**, desenvolvendo pessoas do time e sendo a referência em tomadas de decisão técnicas . Por exemplo, um SRE Sênior pode liderar a iniciativa de implementar SLOs e dashboards de confiabilidade para todos os serviços críticos da empresa, orientando plenos e juniores no processo.
- **Conhecimento Profundo**: Domina **sistemas operacionais, redes e hardware** a ponto de depurar rapidamente problemas complexos e identificar oportunidades de otimização em arquitetura e código . Entende profundamente como construir sistemas tolerantes a falhas, altamente disponíveis e auto-recuperáveis, aplicando esses princípios tanto em novos projetos quanto melhorando sistemas existentes . Também possui **visão abrangente de**

produto – compreende as necessidades do cliente e as traduza em requisitos técnicos de confiabilidade.

- **Escopo e Impacto**: Tem atuação **além da equipe local**, influenciando padrões e práticas de SRE em toda a organização. Por exemplo, o sênior **lidera a definição das metas de confiabilidade (SLO/SLAs) da equipe ou até de vários serviços】 e prioriza endereçar os principais problemas de confiabilidade identificados . Participa de discussões de arquitetura de novos produtos para garantir que aspectos não funcionais (resiliência, observabilidade, segurança) estejam contemplados.
- **Incidentes e Resposta**: SRE Sênior frequentemente atua como **gerente de incidentes** em ocorrências graves, coordenando resolução em tempo real e condução de *postmortems* de alta qualidade. Tem experiência com **gestão de crises** e comunicação com stakeholders de negócio durante incidentes. Consegue analisar padrões de falhas e propor ações preventivas de longo prazo (como Chaos Engineering ou melhorias arquiteturais).
- **Soft Skills e Cultura**: Além das habilidades técnicas robustas, um sênior deve demonstrar **comunicação clara e influência** – capaz de persuadir times a adotar práticas SRE, "vender" melhorias de confiabilidade para executivos usando linguagem de negócio e dados. Deve também cultivar uma cultura blameless e de aprendizado contínuo. Em resumo, o SRE Sênior é guardião da cultura de **confiabilidade e engenharia de excelência** dentro da organização.

Fundamentos de SRE (Prática Avançada e Estratégia)

- **Livro Engenharia de Confiabilidade do Site - Google**: Releituras selecionadas do livro original focadas em tópicos avançados:

  - *Capítulo "Gerenciamento de Incidentes"* e *"Resposta de Emergência"* – abordam em detalhes funções de Incident Commander, comunicação em incidentes, rotinas de on-call, etc. Importante para seniors refinarem seu processo de incidentes .
  - *Capítulo "Abraçando o Risco"* – discute cálculo de risco, decisões de trade-off entre novas features vs. estabilidade. Ajuda o sênior a tomar decisões informadas sobre quando frear lançamentos devido a error budgets estourados .
  - *Capítulo "Eliminando o Trabalho Repetitivo (Toil)"* – fornece estratégias para identificar atividades manuais repetitivas e priorizar automação . SRE Sênior deve liderar iniciativas de redução de toil, e esse capítulo guia quanto esforço investir para obter retorno significativo.

- **Site Reliability Workbook (Capítulos avançados)**: Este livro traz capítulos específicos extremamente relevantes:

  - *"Escalando Práticas SRE"* – discute como evoluir a função SRE conforme a organização cresce (estratégias de engajamento SRE vs. dev, modelos de equipe). Importante para um sênior/arquitetura decidir como posicionar SREs em várias equipes (no *Framework Stone*, papéis Senior/Principal SRE incluem responsabilidade por estratégia técnica de áreas inteiras).
  - *"Práticas de Capacity Planning"* – SREs seniores muitas vezes conduzem planejamento de capacidade e gestão de custos vs. confiabilidade. Buscar materiais sobre **previsão de demanda e capacidade** (talvez Google SRE Workbook ou Google Cloud Architeture Framework em pt).
  - *"Chaos Engineering"* – capítulo ou material externo que discuta testes de falhas proativos. Embora não explicitamente mencionado na pergunta, faz parte de estratégias avançadas de confiabilidade. Há conteúdo em português (palestras em eventos) sobre Chaos Engineering – ex.: **"Chaos Engineering na prática – Netflix Simian Army"** (legendado ou conteúdo local), bom para inspirar criação de exercícios de caos controlado no ambiente.

- **Artigos/Papers de Indústria**: Um SRE Sênior deve se manter atualizado com tendências:

  - **State of DevOps Report (DORA)** – disponível em português resumido. Oferece insights sobre métricas de elite (tempo de recuperação, frequência de deploy, etc.) e práticas de alto desempenho. Isso ajuda o sênior a orientar a empresa rumo a maturidade DevOps/SRE com dados.
  - **Artigo "Beyond the 4 Golden Signals"** – discussões sobre monitorar experiência do cliente, SLIs de alta qualidade. Talvez Atlassian ou Datadog tenham blog PT a esse respeito.
  - **Google SRE lessons learned**: Se possível, procurar por conteúdo traduzido como *"Lições aprendidas de outras indústrias"* (há capítulo no livro SRE) , mostrando como aplicar práticas de confiabilidade de aviação, automotivo, saúde em TI.

- **Comunidade e Casos**: Participar de comunidades SRE:

- Acompanhar meetups ou conferências (muitas tiveram edições online em 2021-2023 com conteúdo em português). Por exemplo: *SREcon22 em português*, *DevOpsDays*, *Meetup SRE Brasil*. Nesses eventos, profissionais seniores compartilham casos do mundo real, como "Implementando SRE no iFood", "Escalando infra do Nubank" etc. Os vídeos/gravações muitas vezes ficam disponíveis e contêm aprendizados valiosos em nosso idioma.
- **Estudo de Caso**: *"Histórias de um SRE"* – a Red Hat disponibilizou um e-book gratuito em português com uma coleção de histórias de SREs seniores (experiências pessoais) . Ler essas histórias dá perspectiva humana e prática do trabalho de um SRE sênior, incluindo erros cometidos e soluções encontradas.

Tecnologias Essenciais (Linux, Docker, Kubernetes, AWS, Terraform, Ansible)

- **Linux (Expertise e Especialização)**: O SRE Sênior muitas vezes se torna especialista em determinados aspectos:

  - **Depuração Profunda**: Aprender uso avançado de ferramentas como *strace*, *perf*, *bcc/eBPF tools* (ex.: *bpftrace*). Há webinars em português sobre eBPF (p.ex. *"Observabilidade com eBPF"* pelo Núcleo de Informação e Coordenação do Ponto BR - Início) que mostram como inspecionar sistemas em baixo nível em produção sem overhead significativo.
  - **Kernel Tuning e Debug**: Ler documentação e posts (mesmo que em inglês, pois em português são raros) sobre parâmetros de kernel (VM swappiness, file descriptors tuning, network sysctls). Entretanto, há conteúdo traduzido, como sessões do Linux Performance Analysis in 60s (Brendan Gregg) adaptadas para PT.
  - **Automação de Admin**: Um sênior deve liderar a configuração de milhares de máquinas via código. Se sua infra é híbrida (VMs e containers), possivelmente precisa orquestrar *PXE boot/Kickstart, Ansible provisioning bare-metal* etc. Buscar conteúdos específicos conforme seu contexto – ex.: *"Automatizando instalação de servidores Linux com Ansible - case Globo*

- **Docker/Containers (Enterprise scenarios)**:

- **Registry e Imagem Enterprise**: Saber administrar um registro privado de imagens (Artifactory, Harbor, ECR, etc.). Talvez criar políticas de image scan e promoção de imagens entre ambientes – isso aparece em blogs corporativos. Sênior deve garantir supply chain segura de containers.
- **Orquestração além do Kubernetes**: Conhecer *Docker Swarm* ou *Nomad* da HashiCorp (há menos material PT, mas conhecer por alto). Isso amplia a compreensão de diferentes trade-offs e cenários legados.
- **Container Networking & Storage**: Conteúdos avançados sobre networking de containers (ex.: *"Entendendo CNI e plugins de rede no Kubernetes"* – alguns materiais em pt-BR existem) e sobre soluções de storage distribuído para containers (como Portworx, Ceph). Um SRE Sênior envolvido com containers em produção precisa tomar decisões informadas sobre esses aspectos.

- **Kubernetes (Expert e Arquiteto de Cluster)**:

  - **Desenho de Arquitetura**: Capacidade de projetar uma **plataforma Kubernetes escalável**. Isso inclui escolher corretamente tipos de nós, dimensionar clusters, decidir estratégia multi-cluster vs. cluster único multi-tenant, planejar upgrades sem downtime etc. Fontes: *Kelsey Hightower's Kubernetes Hard Way* (não PT, mas há traduções não oficiais), e **documentos da CNCF** – talvez o *"Kubernetes Architecture"* tenha alguma versão PT ou resumos em blogs brasileiros.
  - **Operadores (Operators)**: Entender o padrão de Operators para automatizar operações. Já existem conteúdos em pt (por exemplo, um artigo *"Criando um Operator Kubernetes em Go – Introdução"*). Mesmo que não desenvolva um do zero, sênior deve conseguir avaliar e utilizar operators de terceiros para banco de dados, caches, etc.
  - **Segurança em K8s**: Aprofundar temas de segurança: Network Policies (há tutoriais PT no Medium DEV Community), RBAC completo, escaneamento de imagens e uso de admission controllers (OPA/Gatekeeper). Materiais possivelmente em português: meetups sobre *Kubernetes Seguro* ou blogs de empresas (concrete, chaordic) que compartilharam suas práticas.
  - **Ferramentas avançadas**: Experimentar **Service Mesh** (Istio, Linkerd) – muitos conceitos podem ser aprendidos de artigos em português (ex.: *"Introdução ao Istio Service Mesh"* – tradução no site da Istio ou blog da Red Hat). Isso prepara o sênior para ambientes de microserviços complexos.

- **AWS (Arquiteto Sênior)**:
  - **Certificação AWS Architect Professional** – estudar os tópicos (mesmo sem certificar) fornece visão de arquiteturas complexas: multi-region active-active, designs para disaster recovery, otimização de custos em larga escala. Há menos conteúdo em português para o nível Professional, mas alguns cursos pagos oferecem versões PT. Alternativamente, ler whitepapers da AWS (muitos disponíveis em português europeu/brasileiro) como *"AWS Well-Architected Framework"* completo, *"Reliability Pillar"* e *"High Availability Architecture"*.
  - **Serviços Avançados**: Dominar serviços além do básico: ex. *AWS Lambda e arquitetura Serverless*, *Amazon EKS (Kubernetes gerenciado)*, *AWS Organizations* (para multi-conta), *Service Catalog/Terraform Cloud integration*. Muitos webinars e blogs PT da AWS cobrem esses assuntos (procure séries como *"Construindo Arquiteturas Serverless – AWS em Português"*).
  - **FinOps na AWS**: Um SRE Sênior cuida também de eficiência de custos. Buscar conteúdo PT sobre otimização de custos AWS (por ex., *"Como reduzir custos EC2 e EBS"* – blogs de consultores AWS brasileiros). Isso ajuda a tomar decisões de confiabilidade vs. custo (e.g., manter instâncias reservadas para certos serviços críticos).
  - **Casos de Falhas Reais**: Ler/ouvir *postmortems de falhas da AWS* (muitos traduzidos em artigos de notícias tech) para entender pontos de falha e mitigação. Ex.: incidente S3 2017, ou a queda de us-east-1... SREs seniores aprendem muito analisando grandes incidentes públicos e tirando lições para suas próprias arquiteturas.
- **Terraform (Arquitetura de Infra as Code)**:
  - **Organização em larga escala**: SRE Sênior deve definir padrões para todo time seguir em IaC. Uma leitura recomendada é o *"Terraform Guidebook – Collaborative IaC"* (se existir tradução ou resumo) ou posts detalhados sobre structurar Terraform em repositórios monorepo vs. multi-repo, separar estados por ambiente, etc. O *Terraform Best Practices (PT)* já dá direcionamentos valiosos nesse sentido .
  - **Terraform Enterprise/Cloud**: Entender as ferramentas gerenciadas. Terraform Cloud tem documentação em português parcial. Mesmo se não usar, saber as features (remote runs, policy as code com Sentinel) ajuda a criar soluções internas similares com open source.

- **Automatização e Pipelines**: Integrar Terraform em pipelines complexos, garantindo *plan/apply* seguros. Talvez escrever *tests automatizados para código Terraform*: ex., usar *Terratest* (tests em Go) – documentação e exemplos geralmente em inglês, mas há quem blogou em português sobre escrever testes para módulos Terraform.
- **Multi-cloud/híbrido**: Terraform permite orquestrar recursos em vários provedores. Um SRE Sênior deve pelo menos saber criar recursos básicos no Azure e GCP via Terraform. Hashicorp Learn tem labs – aproveitar que a interface permite selecionar português e seguir tutoriais cross-cloud.

- **Ansible (Automação em Escala e Stack Completa)**:

  - **Ansible em ambientes complexos**: Ex: usar Ansible para gerenciar clusters Kubernetes (há playbooks para isso), ou combinar Ansible e Terraform (Terraform cria infra, Ansible configura dentro das máquinas). Alguns blogs PT demonstram casos – *"Provisionando AWS com Terraform e configurando tudo com Ansible"*.
  - **Estratégias de Deploy com Ansible**: Por exemplo, deploy azul-verde usando playbooks, ou rolling update customizado para aplicações stateful. Conteúdo possivelmente encontrado em apresentações de empresas (Busca por *"Ansible deploy Blue Green"* PT, se existir).
  - **Contribuir/Open Source**: Como especialista, seria interessante contribuir em projetos ou roles Ansible públicas, ou escrever artigos. Essa prática consolida o conhecimento. (Por exemplo, publicar no Medium um tutorial avançado em português ajuda a comunidade e reforça seus próprios conhecimentos.)

Observabilidade e Monitoramento (Prometheus, OpenTelemetry, New Relic)

- **Prometheus (Escalabilidade e EcoSistema)**:

  - **Thanos/Cortex**: Um SRE Sênior deve avaliar soluções para escalar Prometheus e retenção longa de métricas. Projetos como *Thanos* ou *Cortex* (monitoramento distribuído) são usados em grandes empresas. Buscar conteúdo PT-BR sobre "Escalando Prometheus com Thanos" – há talks legendadas e alguns blogs (talvez comunidade Zup ou DTI).
  - **Métricas de Negócio**: Além de métricas técnicas, liderar a instrumentação de métricas de produto. Orientar times dev a expor métricas customizadas (como taxa de conversão, etc.) e coletá-las no Prometheus. Um bom SRE Sênior sabe que confiabilidade também é

entregar valor ao usuário final. Artigo Atlassian (EN/PT) *"Measuring SLIs that matter to users"* – verificar se há versão PT.

- **Operação de monitoramento**: Estabelecer processos de tuning de alertas para evitar alerta em demasia (noise). Implementar revisão periódica de alertas SRE. Ferramentas tipo *Alertmanager + Jira integration*para rastrear débitos técnicos – possivelmente case interno, mas pode inspirar ouvir podcasts ou entrevistas de SREs seniors (ex.: *"Alert Fatigue – how we solved at X company"*, se disponível).
- **Observabilidade Holística**: SRE Senior integra diferentes sinais – pode correlacionar métricas do Prometheus com traces do OpenTelemetry e logs. Portanto, além de PromQL, conhecer **Grafana Loki** (logs), **Grafana Tempo** (traces) – stack "PLG". A Grafana Labs tem webinars PT sobre essa stack. Esse conhecimento permite ao especialista projetar soluções open-source completas de observabilidade (sem depender sempre de suite paga).

- **OpenTelemetry (Padrões Organizacionais)**:

  - **Governança de Telemetria**: No nível especialista, espera-se definir padrões de instrumentação para todos os desenvolvedores. Por exemplo, decidir quais atributos de trace devem ser padrão em todos os serviços (correlation IDs, user IDs anonimizados, etc.). Criar um *manual de observabilidade* interno. Para isso, pode-se inspirar em material PT como o *Guia de Observabilidade do OpenTelemetry* (site já mencionado) e complementá-lo com necessidades específicas do negócio.
  - **Análise de Tracing Avançada**: Saber usar trace para achar *bottlenecks* complexos (tipo *fan-out em microserviços*). Ferramentas: Jaeger UI ou New Relic APM trazem essa capacidade – um especialista deve extrair informações profundas, como identificar o microserviço que mais impacta latência p95 de uma transação complexa. Se precisar, buscar estudo de caso – ex: *"Como o Uber usa tracing distribuído"* (há posts no blog do Uber traduzidos).
  - **Contribuição na Comunidade**: Um especialista/arquitetura pode participar da comunidade OpenTelemetry (mesmo que internacional). Porém, localmente, pode apresentar lightning talks em meetups para difundir instrumentação – isso reforça seu domínio e networking.

- **New Relic (Observabilidade de Empresa)**:

- **Arquitetura Observabilidade**: O especialista SRE avalia soluções enterprise – por exemplo, pode comparar New Relic vs. Datadog vs. Grafana Enterprise para uma decisão corporativa. Deve entender custos, facilidade de implementação, integração com OpenTelemetry, etc. Relatórios de analistas (Gartner, etc.) frequentemente têm versões em português resumidas que podem ajudar. Ou então eventos locais de parceiros (por ex, Meetup New Relic Brasil).
- **Customização Extrema**: Em New Relic, um especialista deve saber criar **alertas sintéticos** (simular transações do usuário), configurar **dashboards executivos** com KPIs de SRE (SLOs atingidos, número de incidentes, MTTR). Muitas dessas ações envolvem definir queries NRQL combinando dados de APM, Infra e custom events. A documentação da New Relic tem seção "NRQL Examples" (não em PT, mas code-oriented).
- **Integrar New Relic com Open Source**: Um desafio comum – usar New Relic junto com Prometheus/OTel. O especialista pode escrever plugins ou usar exporters para alimentar o New Relic (existe *Prometheus OpenMetrics integration* na NR). Seguir guias (em inglês) e talvez escrever documentação interna em português para outros times sobre como usar esses fluxos de dados.
- **ROI de Observabilidade**: Em papéis arquiteturais, muitas vezes é preciso justificar investimento. Preparar conteúdo (em PT) para diretoria mostrando ganhos obtidos com observabilidade (menos downtime, resolução mais rápida). Isso não é um conteúdo técnico específico, mas uma habilidade que o especialista SRE desenvolve – e recursos para isso podem ser whitepapers traduzidos (*"The Business Value of SRE"* etc.).

Pipelines e CI/CD (Jenkins e DevOps Enterprise)

- **Jenkins (Expert e Ecosistema)**:
  - **Arquitetura e Manutenção**: O especialista pode assumir a arquitetura da solução CI/CD inteira. Isso envolve alta disponibilidade do Jenkins (usar *Jenkins HA* com instâncias redundantes ou standby, backup/restore plan), isolação de builds críticos, etc. Conteúdo para isso pode ser escasso em PT, mas talvez casos de grandes empresas – ex: *"Como o Yahoo escalou 6000 agentes Jenkins"* (se tiver resumo PT).

- **Pipelines Complexos**: Projetar pipelines multi-stage incluindo *gates manuais*, integração com testes de performance e segurança automatizados. Procurar em português algo como *"Pipeline CI/CD enterprise"* ou materiais da Azure DevOps (apesar de outra ferramenta, os conceitos de gates e releases contínuos podem ser aproveitados).
- **CI/CD Guild**: O especialista SRE muitas vezes lidera uma *guilda* ou capítulo de CI/CD dentro da empresa, definindo padrões que todos os times seguem. Por isso, manter-se atualizado com tendências: ex., adoção de **GitOps** (ferramentas como ArgoCD – existem webinars PT), contêinerização do próprio processo CI (Jenkins dentro de K8s via Jenkins Operator – documentos do Jenkins Operator não estão em PT, mas posts de blog nacionais podem ter experimentos).
- **Quality Engineering Integration**: DevOps/SRE seniors colaboram com Quality Engineers para incorporar testes automatizados de todos os tipos (unit, integration, end-to-end) na pipeline. Avaliar ferramentas como **Chaos Monkey** (para caos) ou **Selenium/Grid** (para testes web) integradas ao Jenkins. Há casos PT de "CI pipeline incluindo testes de carga com JMeter" – isso ilustra como ampliar pipelines para cobrir confiabilidade antes da produção.

- **Ferramentas Modernas de CI/CD**:

  - **ArgoCD/GitOps**: Se a empresa caminha para GitOps, o SRE Arquiteto deve liderar essa implementação. Materiais recomendados: *Livro "GitOps Cookbook"* (se disponível em PT) ou posts da AWS/GCP sobre GitOps em português. O importante é compreender e ensinar a mentalidade de pipelines declarativos, auditáveis via Git.
  - **Tekton**: Pipeline Kubernetes nativo (projeto CNCF). Pouco conteúdo em PT, mas vale conhecer para futuramente talvez migrar para soluções cloud-native de CI/CD.
  - **CI/CD Security (DevSecOps)**: Implementar controles de segurança nas pipelines – *scan de imagem Docker, scan de dependências* (OWASP Dependency Check), etc. Em português, buscar *"DevSecOps pipeline exemplos"*. A TrendMicro, Checkmarx e outras às vezes publicam e-books PT sobre isso. O especialista SRE deve garantir que a esteira de entrega de software seja **confiável e segura**.

SRE Especialista/Arquiteto

*(Nota: Este nível refere-se ao profissional de **nível Staff/Principal SRE ou Arquiteto de Confiabilidade** – alguém que está no topo da carreira técnica de SRE.)*

Habilidades e Conhecimentos Esperados

- **Visão Geral**: O Especialista/Arquiteto SRE atua definindo a **estratégia técnica de confiabilidade** de toda uma área ou de toda a empresa . É responsável por decisões arquiteturais críticas, abrangendo múltiplos sistemas e equipes, garantindo que a confiabilidade e observabilidade estejam incorporadas desde o design. Lida com os pontos mais **críticos e complexos** – aqueles problemas únicos que exigem soluções inovadoras . Seu trabalho tem impacto direto nos objetivos de negócio de longo prazo, influenciando roadmaps de produtos e plataformas.
- **Liderança e Influência Organizacional**: Este profissional frequentemente não possui autoridade hierárquica, mas exerce uma forte **liderança técnica transversal**. Ele influencia direções de equipes inteiras, podendo **encerrar iniciativas que não agregam valor ou redefinir metas** se necessário para alinhar à estratégia da empresa . Atua como mentor de outros líderes técnicos – desenvolve outros SREs seniores e engenheiros, propagando cultura e conhecimento. Cria um ambiente de confiança e incentivo à experimentação, empoderando times a inovar em soluções de confiabilidade .
- **Expertise Técnica e Inovação**: Espera-se conhecimento **profundo e atualizado** em praticamente todas as áreas relacionadas: arquitetura de sistemas distribuídos, bancos de dados, redes globais, segurança, linguagens de programação, etc. O especialista/arquitetura SRE está sempre à frente em pesquisar e adotar novas tecnologias que melhorem confiabilidade (por ex., arquiteturas sem servidor, malhas de serviço, monitoramento com AI Ops). Ele é capaz de **antecipar desafios futuros** de escala e planejar soluções antes que virem problemas .
- **Visão de Negócio**: Diferentemente dos níveis anteriores, um arquiteto SRE tem forte entendimento do contexto de negócio. Sabe **priorizar projetos de SRE alinhados à estratégia da empresa** e aos requerimentos de clientes . Consegue traduzir requisitos de negócios em metas técnicas de confiabilidade e vice-versa – comunicando o estado de confiabilidade em termos de impacto no cliente e receita. Participa ativamente de planejamento de produto, garantindo que novos produtos sejam projetados com observabilidade e resiliência desde o início.

- **Governança e Padrões**: Define padrões globais de engenharia (runbooks, playbooks, práticas de revisão pós-incidente) e assegura aderência. Cria processos para escalar a cultura – por exemplo, um **Centro de Excelência SRE** ou programas de treinamentos internos. Pode também representar a empresa externamente em fóruns de SRE, contribuindo para a comunidade.
- **Resumo**: O Especialista/Arquiteto SRE é quem *"enxerga o todo"*. Ele garante que a empresa esteja **fazendo as coisas certas da maneira certa** em termos de confiabilidade. Resolve os problemas mais espinhosos, orienta dezenas de times e constrói uma visão de longo prazo onde a confiabilidade se torna vantagem competitiva da organização.

Fundamentos de SRE (Estratégia e Liderança)

- **Livros e Publicações de Referência**: Além dos já citados, alguns livros estratégicos:
  - *"The SRE Workbook"* – últimas seções sobre evolução do engajamento SRE, cultura organizacional e lições de outras indústrias . Ajuda a estruturar iniciativas como *Error Budget Policies* organizacionais, programas de treinamentos.
  - *"Engineering Management for Reliability"* (capítulos possivelmente dentro do SRE Workbook ou blogs do Google). Fornece insights de liderança: como estruturar times de plantão globais, motivar engenheiros a pensar em confiabilidade, etc.
  - **Livro "Cloud Reliability Engineering"** (hipotético – consolidando práticas modernas, se disponível).
- **Publicações Acadêmicas/Whitepapers**: O arquiteto deve ler pesquisas e whitepapers:
  - Ex.: **"Site Reliability Engineering at Google – 10 Years Later"** – retrospectiva (se publicada). Ou *"SRE vs DevOps"* por autores reconhecidos (Liz Fong-Jones, etc.). Alguns desses materiais podem não ter tradução, mas vale o esforço ler em inglês e, se possível, preparar resumos em português para mentorar outros.
  - **RFCs Internos e Padronizações**: Criar documentos formais internos definindo *SLO global targets*, *resiliência mínima requerida* por classe de serviço, etc. Não há fontes públicas para isso, mas pode-se basear em exemplos do Google (chapter *"SLOs Globais"* se existisse) ou buscar *"SRE Charter"* de alguma empresa que tenha divulgado.

- **Comunidade Global**: Participar dos eventos internacionais (SREcon, DevOps Enterprise Summit) e depois disseminar. Por exemplo, assistir a uma palestra do SREcon (mesmo inglês) e então apresentar um *brown bag* em português para a engenharia interna, multiplicando conhecimento.
- **Mentoria e Desenvolvimento de Carreira**: Conteúdos sobre *coaching* e *mentoring* são úteis – embora não técnicos, são parte do papel. Um arquiteto SRE precisa cultivar novos líderes. Poderia buscar materiais (artigos PT ou livros) sobre mentoria técnica e liderança servidora, contextualizando em SRE.
- **Inovação Contínua**: Estar à frente em temas emergentes:
  - *AIOps* (Inteligência Artificial para operações) – investigar se publicações da Gartner ou blogs PT da IBM, por exemplo, explicam e dão cases. Ver o que se pode adotar realisticamente para melhorar detecção de anomalias, etc.
  - *Gremlin/Chaos Engineering avançado* – por ex, implementar *GameDays* regulares. Gremlin (ferramenta de caos) tem materiais e webinars em português ou legendados.
  - *Resiliência de negócios* – além de sistemas. Pensar em *BCP/DR (Business Continuity Planning & Disaster Recovery)*. Um arquiteto SRE pode liderar exercícios de DR em escala corporativa. Instruções e normas (ex ISO 22301) muitas vezes têm resumos em português.
- **ROI e Comunicação Executiva**: Aprender a falar a língua de executivos. Sugestão: conteúdos da *Google Cloud Architecture Framework – Reliability* (tem uma versão PT-BR no site da Google Cloud) que foca em KPIs de negócio. Ou artigos tipo *"Making the Business Case for SRE"*. Com isso, formular relatórios trimestrais de SRE para diretoria, mostrando tendências de disponibilidade, sucesso das iniciativas (ex: "reduzimos incidentes P1 em 30% após X projeto"), justificando investimentos.

Tecnologias Essenciais (Visão de Arquitetura e Futuro)

- **Tecnologias de Futuro**: O arquiteto SRE deve mapear novas tecnologias que podem trazer vantagem:
  - **Serverless e FaaS**: Por ex, avaliar se adotar *AWS Lambda* ou *Google Cloud Run* melhora confiabilidade e reduz toil (sem servidores para gerenciar). Encontrar estudos (PT se

possível) ou casos de adoção serverless em empresas grandes (ex.: Magazine Luiza migrando algo para serverless – se houver conteúdo).

- **Arquitetura Orientada a Eventos**: Talvez movimentar a empresa para microserviços event-driven para isolamento de falhas. Ler livros como "Enterprise Integration Patterns" (tem tradução) e adaptar para resiliency (usar filas para suavizar picos, etc.).
- **Edge Computing e CDN**: SRE arquitetos consideram entregar conteúdo de forma mais próxima do usuário para aumentar confiabilidade/performance global. Aprender sobre redes CDN (Cloudflare tem blog em português explicando detecção de falhas e rerouting, por ex.).
- **Computação Híbrida/MultiCloud**: Garantir que sistemas possam rodar em mais de um provedor ou on-premise e cloud (para evitar vendor lock-in ou aproveitar pontos fortes de cada). Estratégias multi-cloud são debatidas em whitepapers – buscar algo como *"Multi-cloud reliability strategies"* (talvez palestras da CNCF; se não em português, resumir internamente).
- **Ferramentas Internas**: Um arquiteto SRE muitas vezes impulsiona o desenvolvimento de ferramentas custom internas (porque as de mercado não suprem totalmente as necessidades). Ex: criar uma ferramenta interna de *change management automatizado*, ou aprimorar a plataforma de incidentes. Para inspiração: *"How Google Runs Production – automation tools"* (conteúdo público limitado). Entretanto, há blogs de empresas brasileiras compartilhando ferramentas caseiras – ex.: Nubank open-source o *Hermes* (talvez fictício) para orquestrar deploy. Vale ficar atento a essas histórias em meetups.

- **Governança de Dados e Compliance**: Como arquiteto, envolver-se também em confiabilidade de dados:

  - Garantir backups, consistência, estratégias de migração. Pautar discussões de "data reliability". Fontes: livros ou cursos de *Data Engineering* (alguns em PT) para entender sistemas de dados distribuídos (Kafka, Cassandra, etc.) sob perspectiva de confiabilidade.
  - Compliance: LGPD, PCI, etc. No trecho do Stone Framework, nota-se que já no nível SRE I menciona conformidade regulatória . O arquiteto SRE deve assegurar que práticas de SRE atendam auditorias – ex: políticas de retenção de log, controle de acesso. Familiarizar-se com esses padrões (muitos materiais em português do governo ou consultorias explicando exigências).

- **Observabilidade Total**: Abrangendo User Experience e Negócio:

  - Implementar **SRE de Negócio** – alguns arquitetos criam o conceito de *"Error Budget de features"* ou monitoramento de KPIs de produto sob a ótica SRE. Isso é bem avançado e pouco documentado, mas a ideia é usar habilidade SRE para melhorar confiabilidade da *experiência do usuário e da entrega de valor*. Por exemplo, métricas de sucesso de login, transações completadas. Esse campo cruza com observabilidade de produto (Product Ops). Buscar se há conteúdo PT sobre *"Product Reliability"*.
  - **AI e Observabilidade**: Experimentar tools de AI Ops que usam machine learning para detectar anomalias em métricas/traces (Datadog Watchdog, New Relic Alerts AI). O arquiteto avalia se vale incorporar para reduzir tempo de detecção. Alguns webinars PT de vendors mostram isso na prática.

- **Cultura Organizacional**: No topo, a tecnologia serve à cultura e pessoas:

  - **Transformação DevOps/SRE**: O arquiteto SRE frequentemente é sponsor de transformação cultural. Livros como "Accelerate" (Forsgren/Kim) têm versão em português e fornecem bases científicas para mudanças. Ele pode aplicar conceitos do livro – e, por exemplo, adotar OKRs de melhoria de confiabilidade organizacional.
  - **Custo da Inatividade**: Ensinar a org sobre custo de downtime com números. Montar planilhas e casos (há artigos PT falando "1 hora fora do ar custou X milhões..." para empresas conhecidas). Isso eleva a prioridade de SRE no nível executivo.
  - **Trabalho com Times de Produto**: Criar um *selo de confiabilidade* para lançamentos – p.ex., nenhum novo serviço vai para prod sem cumprir checklist SRE (monitoramento, SLO definido, runbook criado, etc.). O arquiteto SRE deve elaborar essa checklist (inspirada nas *production readiness reviews* do Google). Pode consultar o *Capítulo "Launch Coordination Checklist"* do livro SRE (Apêndice) disponível em português para não esquecer itens.

Observabilidade e Monitoramento (Estratégia e Big Picture)

- **Plataforma de Observabilidade Unificada**: O arquiteto SRE pode impulsionar a construção de uma **plataforma unificada** que agregue métricas, logs, traces e talvez usuário real monitoring. Isso pode envolver integrar diversos sistemas (Prometheus + Elastic + Jaeger +

UI custom). Há casos notórios: LinkedIn's in-house monitor (não PT) ou Netflix Atlas. Mas no Brasil, talvez Nubank, PicPay, tenham palestras de como montaram suas plataformas – acompanhar essas histórias fornece direcionamentos.

- **Indicadores de Negócio e SLOs Corporativos**:
  - Montar **SLOs a nível produto ou empresa**. Ex.: "99.9% das transações de pagamento devem completar com sucesso em trimestre". Isso requer consolidar vários SLIs técnicos. O arquiteto SRE lidera esse esforço e reporta periodicamente para executivos. Ferramenta: implementar um *SLO Dashboard corporativo*. Pode usar algo open source tipo *Sloth* (framework SLO) ou Google Cloud Service Monitoring (há docs PT).
  - **Error Budget Policy**: Estabelecer políticas organizacionais para ações quando budgets estouram (ex: congelar deploys). Precisa apoio executivo; o arquiteto SRE cria propostas embasadas e orienta squads a seguir. Referência: *Chapter "SLOs and Error Budget Policies"* (SRE Workbook) – provavelmente não há em PT pronto, mas o arquiteto pode escrever material interno adaptado.

- **Resiliência e Chaos Engineering corporativos**:
  - Institucionalizar **GameDays**: O arquiteto SRE organiza eventos periódicos em que falhas simuladas são injetadas e as equipes treinam resposta. Documentos para isso – a empresa Gremlin disponibiliza um *"Chaos Engineering Cookbook"* (talvez traduzido).
  - Avaliar ferramentas chaos enterprise (Gremlin, Litmus) – conversar com fornecedores (em PT) para entender se agregam valor além de scripts caseiros.
  - Criar uma cultura onde falhas são vistas como oportunidades de aprendizado (e o arquiteto garante que orçamentos de tempo sejam dedicados a isso).

- **Benchmarking e Continuidade**:
  - *Benchmarking externo*: Comparar confiabilidade da sua empresa com a de concorrentes ou líderes. Se houver índices públicos (p. ex, uptime de grandes serviços via downdetector ou relatórios de mercado), use-os. Pode também realizar *peer reviews* informais com arquitetos de outras empresas (comunidades DevOps).

- **Sustentabilidade da Operação**: Garantir que a escalada de complexidade não sobrecarregue SREs. O arquiteto deve acompanhar métricas de *toil* (ex: % do tempo gasto em atividades operacionais) e liderar automações para manter isso controlado. Esse conceito é do livro SRE; traduzido, possivelmente se refere a "trabalho pesado" ou "esforço manual" .
- **Planos de Sucessão e Mentoring**: Por fim, a meta de um Especialista/Arquiteto é fazer com que a confiabilidade *não dependa de indivíduos mas de processos e cultura*. Então ele documenta, treina e prepara outros para que a empresa continue confiável mesmo se ele sair. Isso está implícito em culturas como da Stone, onde se diz "*sabe que só existe crescimento com sucessão*" – aplicável aqui também.

Pipelines e CI/CD (Nível Organizacional e Inovação)

- **Arquitetura de DevOps Platform**: Muitas empresas com vários times optam por criar uma **Plataforma DevOps interna** (pense em algo como "Paved Road" da Netflix). O arquiteto SRE provavelmente liderará isso:

  - Construir um portal self-service onde devs podem criar novos repositórios já com pipeline configurada, monitoramento padrão, etc. Ferramentas open source como *Backstage* (da Spotify) ajudam – felizmente há tradução do Backstage docs e posts PT sobre adoção (ex.: Youse, Zup).
  - Automatizar integrações: ex, ao criar um microserviço novo, pipeline CI/CD é criada automaticamente no Jenkins/GitHub Actions via templates. Escrever esses templates e mantê-los.
  - Permitir **Dev Prod Parity**: pipelines que facilitam replicar ambiente prod em dev (via containers, etc). Pesquisar *"plataforma de desenvolvimento interna case"*.

- **CI/CD de Próxima Geração**:

  - Acompanhar evolução: ex, *CI/CD GitOps total*, *pipelines declarativas universais*, *CI/CD in-browser IDE*. Esses conceitos aparecem em blogs de vanguarda (Jenkins X, Harness, etc). O arquiteto avalia se e quando adotar.
  - **Metrics de Pipeline**: Instrumentar as pipelines – medir tempo de build, frequência de implantações, taxa de falha de deploy. Criar SLOs para o *sistema de delivery* (por ex: "90%

dos pipelines completam em < X minutos”). Isso melhora a eficiência eng. e é responsabilidade difusa entre Dev e SRE – mas o arquiteto SRE pode puxar isso.

- **DevSecOps e Compliance Integrada**:
  - Em nível enterprise, pipelines também devem garantir conformidade. O arquiteto SRE trabalha com Security Team para incorporar verificações automáticas em cada fase (scan estático, análise de vulnerabilidades containers, testes de invasão automatizados).
  - Ferramentas como *Checkov*, *Trivy*, *SonarQube* – há conteúdo PT (blog da PayGo sobre SonarQube, etc.). Montar esse *pipeline de segurança* corporativo e evangelizar.
- **AI/ML Ops**: Se a empresa tem produtos de Machine Learning, o arquiteto SRE pode se envolver em **MLOps pipelines** (que são diferentes, envolvendo preparação de dados, treinamento modelos, etc). Poucas empresas têm isso em PT ainda, mas comunidade de Data Science Brasil discute. Adquirir ao menos noções para suportar times de Data eng/ML eng sob perspectiva de confiabilidade (ex: garantir que retraining não derruba sistema, etc).
- **Case Studies e Networking**:
  - Em suma, para este nível, além de cursos (que são raros), o aprendizado vem de **networking com outros arquitetos**. Participar de grupos como *SRE Lead Google Group* (se tiver), ou eventos executivos. No Brasil, buscar grupos de discussão (talvez WhatsApp/Telegram de DevOps/SRE com gente sênior).
  - **Publicar Conteúdo**: Tornar-se também um influencer. Escrever artigos avançados (em português e inglês) sobre soluções inovadoras que implementou. Apresentar em conferências locais (DevOpsDays) – isso consolida sua autoridade e mantém você afiado, além de contribuir de volta para a comunidade.

SRE Júnior - Fundamentos e Primeiros Passos

Fundamentos de SRE:

- Webinar: Introdução ao SRE - Google Cloud
- Livro SRE do Google em Português - ElvenWorks
- Artigo: SLI, SLO, SLA e Error Budget explicados
- Guia Prático de SRE (ElvenWorks)

- Curso Coursera: Desenvolvendo Cultura SRE Google - Grátis como ouvinte

Tecnologias:

- Curso Linux - Primeiros Passos (Curso em Vídeo)
- Livro: Descomplicando Docker (Jeferson Fernando - LINUXtips)
- Curso Intensivão Docker 2h - YouTube
- Curso Docker Fundamentals - DIO
- Livro Descomplicando Kubernetes - LINUXtips
- Curso AWS Cloud Practitioner Essentials (PT-BR)
- AWS Educate - Plataforma Gratuita
- Guia Rápido Terraform em Português
- Curso Descomplicando Ansible - YouTube

Observabilidade:

- Curso Prometheus e Grafana - Ciro Lini GitHub
- Introdução à Observabilidade e OpenTelemetry (Site Oficial)
- Artigo: O que é OpenTelemetry? - New Relic PT
- Documentação New Relic em Português

Pipelines:

- Curso Jenkins para Iniciantes - Udemy Grátis
- Playlist Curso de Jenkins - YouTube

SRE Pleno - Consolidação e Profundidade

Fundamentos:

- Livro SRE Workbook (Português) - ElvenWorks
- Artigo: SLA vs SLO vs SLI - Atlassian
- Labs Práticos Google Cloud - SRE

Tecnologias:

- Curso Linux Essentials - Cisco Networking Academy
- Foca Linux - Guia Completo em PDF
- Curso Docker Completo - DIO
- Kubernetes - Descomplicando Kubernetes Avançado
- Curso AWS Solutions Architect - YouTube (PT-BR)

- Terraform Best Practices em Português
- Série Avançada de Ansible - 4Linux

Observabilidade:

- Prometheus Operator - Documentação Oficial
- OpenTelemetry Collector - Guia
- Guia Monitorando Kubernetes com New Relic

Pipelines:

- Documentação Jenkins Pipeline Declarativo
- Curso CI/CD com GitHub Actions - YouTube

SRE Sênior - Referência Técnica e Liderança

Fundamentos:

- Livro SRE Google - Capítulo de Incident Management
- Livro Workbook - Capítulo Scalability of SRE Practices
- Relatório State of DevOps - DORA em Português
- Histórias de um SRE - Red Hat PT

Tecnologias:

- Ferramentas eBPF - Tutorial do BCC
- Kubernetes Security - Medium PT
- AWS Well-Architected Labs (PT-BR)
- Terraform Cloud - Documentação
- Guia Deploys Zero Downtime com Ansible

Observabilidade:

- Thanos - Escalando Prometheus
- Grafana Loki - Logs Centralizados
- Guia SLOs Reais no Grafana
- Case OpenTelemetry no Uber (EN)

Pipelines:

- GitOps com ArgoCD - Guia PT
- Jenkins em Kubernetes - Operador Jenkins

- Integração DevSecOps com Checkov e Trivy

Observação:

Alguns links levam a conteúdos originalmente em inglês, mas a maioria possui tradução ou legendas PT-BR. Onde aplicável, o conteúdo em português foi priorizado.